Practice 07 (2) — Read Replicas and Parameter Groups

Objective: add a **read replica** to a PostgreSQL database and prove it really is one: it answers reads, refuses writes, and catches up with the primary a moment later. Then change engine settings the RDS way, with a **parameter group**, and see which changes need a reboot. These are the skills from today's lecture (Block 6 and Block 7).

This is the **second** practice for week 7. Practice 1 covered the private database, the SG-to-SG rule, TLS and snapshots. This one builds on the same setup, but you don't need to have finished Practice 1.

Timebox: in class: ~50–55 min on Track A, ~35 min on Track B. If you've been stuck on one step for more than 10 minutes, stop and ask. Don't push through alone.

> **How you're checked:** this practice is done in class and checked **orally**. There is no report. When you finish, keep your terminal (and console, on Track A) open. The instructor will ask you to show the items in the *Definition of done* and to explain what you did.

Pick your track

You don't need AWS access to pass. Pick **one** track:

	Track A — real AWS	**Track B — no AWS**
Needs	AWS Academy Learner Lab (or your own account), two waits for RDS	Docker (or Podman) and a bash shell (Linux, macOS, or WSL / Git Bash on Windows)
Database	RDS for PostgreSQL primary + read replica, `db.t3.micro`	Two `postgres:17` containers: primary + streaming replica
Slow steps	Creating the primary (5–15 min) and the replica (5–10 min): **start them early**	none

The ideas are the same on both tracks:

AWS (Track A)	Track B equivalent
Primary DB instance	`course-db` container
Automated backups on (retention ≥ 1 day), required for replicas	`wal_level=replica` (the default) and a `replicator` user with the `REPLICATION` role
`create-db-instance-read-replica`	`pg_basebackup -R` into a new volume, then start `course-db-replica` on it
Replica **endpoint** (its own DNS name)	Container name `course-db-replica`
Asynchronous replication	PostgreSQL streaming replication (`sync_state = async`)
`ReplicaLag` metric	`pg_stat_replication` on the primary, `pg_last_xact_replay_timestamp()` on the replica
DB parameter group	`ALTER SYSTEM SET ...` (writes `postgresql.auto.conf`)
Dynamic parameter, applied immediately	Parameter picked up by `SELECT pg_reload_conf();`
`pending-reboot`	`pending_restart = t` in `pg_settings` , fixed by `docker restart`
`promote-read-replica`	`SELECT pg_promote();`

Definition of done

Be ready to **show** each of these:

- □ A primary and a read replica, each with **its own endpoint** (Track A: two different `Endpoint.Address` values. Track B: two containers, `course-db` and `course-db-replica`).
- □ On the replica: `SELECT pg_is_in_recovery();` returns `t` , a `SELECT count(*)` works, and an `INSERT` fails with `cannot execute INSERT in a read-only transaction` .
- □ Rows inserted on the **primary** appear on the **replica** a moment later.
- □ Replication lag, measured (Track A: SQL on the replica, optionally the `ReplicaLag` metric. Track B: `pg_stat_replication`).
- □ `SHOW log_min_duration_statement;` returns `500ms` on the primary, set through a parameter group (Track A) or `ALTER SYSTEM` (Track B), and you can say which change needed a reboot or restart and why.
- □ Cleanup done and shown.

Track A — real AWS (AWS Academy Learner Lab)

> **Using AWS Academy Learner Lab**

1. Open the **AWS Academy Learner Lab** course → **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to **AWS** turns green (1–3 min), then click **AWS** to open the console.
2. Use region `us-east-1` (N. Virginia). Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed; other regions fail.
3. CLI: use the terminal on the lab page (the AWS CLI is already set up), or copy the credentials from **AWS Details** → **AWS CLI** → **Show** into `~/.aws/credentials` on your laptop. They change every session.
4. Key pair: don't create one. Use the lab's `vockey` : **AWS Details** → **Download PEM** saves `labsuser.pem` .
5. When you're done, clean up (A7) and click **End Lab**.

Per the Learner Lab Readme, RDS is available, but some actions or instance classes may be restricted. If `create-db-instance-read-replica` or `create-db-parameter-group` is refused (`AccessDenied` / `not authorized`), don't fight it: switch to **Track B**, which shows the same ideas.

Order matters here. Start the primary (A2) **first**, launch the app server (A3) while it's being created, and start the replica (A5) as soon as the primary is `available` . Budget about **50–55 minutes**.

A1. Network (short version of Practice 1)

```
export AWS_REGION=us-east-1

VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)

SG_APP=$(aws ec2 create-security-group --group-name lab07b-sg-app \
  --description "lab07b app tier" --vpc-id $VPC --query GroupId --output text)
SG_DB=$(aws ec2 create-security-group --group-name lab07b-sg-db \
  --description "lab07b db tier" --vpc-id $VPC --query GroupId --output text)

MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id $SG_APP --protocol tcp --port 22 --cidr
$MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG_DB --protocol tcp --port 5432 --source-
group $SG_APP

SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC \
  --query 'Subnets[0:2].SubnetId' --output text)
aws rds create-db-subnet-group --db-subnet-group-name lab07b-db-subnets \
  --db-subnet-group-description "lab07b DB subnets" --subnet-ids $SUBNETS
```

A2. Create the primary (start it now)

Automated backups must be **on** (retention at least 1 day). RDS refuses to create a read replica from an instance with backups turned off.

```
PW=$(openssl rand -base64 18 | tr -d '/@" ')
echo "$PW" > .dbpw && chmod 600 .dbpw     # never commit it

aws rds create-db-instance \
  --db-instance-identifier lab07b-db \
  --engine postgres \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 --storage-type gp3 \
  --master-username dbadmin --master-user-password "$PW" \
  --db-name shop \
  --db-subnet-group-name lab07b-db-subnets \
  --vpc-security-group-ids $SG_DB \
  --no-publicly-accessible --no-multi-az \
  --backup-retention-period 1
```

Don't wait here. Go to A3, and come back with:

```
aws rds wait db-instance-available --db-instance-identifier lab07b-db
```

A3. Launch the app server (while the primary is being created)

```
APP_ID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro --key-name vockey \
  --security-group-ids $SG_APP \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab07b-app}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids $APP_ID
APP_IP=$(aws ec2 describe-instances --instance-ids $APP_ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo $APP_IP
```

A4. Load sample data into the primary

Once the primary is `available`, get its endpoint on your laptop:

```
aws rds describe-db-instances --db-instance-identifier lab07b-db \
  --query 'DBInstances[0].[Endpoint.Address,EngineVersion]' --output text
cat .dbpw
```

SSH to the app server (`ssh -i labsuser.pem ec2-user@$APP_IP`) and run:

```
sudo dnf install -y postgresql17 || sudo dnf install -y postgresql16 || sudo dnf install -y
postgresql15
export PRIMARY=<primary endpoint>
export PGUSER=dbadmin PGDATABASE=shop PGSSLMODE=require
export PGPASSWORD='<paste the password once>'

psql -h $PRIMARY -c "CREATE TABLE orders (id serial PRIMARY KEY, amount numeric(10,2), created_at
timestamptz DEFAULT now());"
psql -h $PRIMARY -c "INSERT INTO orders (amount) SELECT round((random()*100)::numeric, 2) FROM
generate_series(1, 5000);"
psql -h $PRIMARY -c "SELECT count(*) FROM orders;"      # 5000
```

A5. Create the read replica (several minutes)

Back on your laptop:

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier lab07b-replica \
  --source-db-instance-identifier lab07b-db \
  --db-instance-class db.t3.micro \
  --vpc-security-group-ids $SG_DB \
  --no-publicly-accessible
```

This takes **5–10 minutes**. While you wait, do **A6** (the parameter group), then come back:

```
aws rds wait db-instance-available --db-instance-identifier lab07b-replica
aws rds describe-db-instances \
  --query 'DBInstances[?starts_with(DBInstanceIdentifier,`lab07b`)].
[DBInstanceIdentifier,Endpoint.Address,ReadReplicaSourceDBInstanceIdentifier]' \
  --output table             # two different endpoints; the replica names its source
```

On the app server (predict each result first):

```
export REPLICA=<replica endpoint>

psql -h $REPLICA -c "SELECT pg_is_in_recovery();"                # t: it's a replica
psql -h $REPLICA -c "SELECT count(*) FROM orders;"               # 5000: reads work
psql -h $REPLICA -c "INSERT INTO orders (amount) VALUES (1);"    # ERROR: cannot execute INSERT
in a read-only transaction

psql -h $PRIMARY -c "INSERT INTO orders (amount) SELECT 42 FROM generate_series(1, 100);"
psql -h $REPLICA -c "SELECT count(*) FROM orders;"               # 5100, maybe after a second

# how far behind is the replica? (time since the last change it replayed)
psql -h $REPLICA -c "SELECT now() - pg_last_xact_replay_timestamp() AS lag;"
```

The lag query shows the time since the replica last **replayed** a change. Right after an insert it's small; on an idle database it keeps growing, which doesn't mean the replica is broken. Optionally, look at the `ReplicaLag` metric for `lab07b-replica` in **CloudWatch → Metrics → RDS** in the console.

A6. A custom parameter group

RDS doesn't let you edit `postgresql.conf`. You change settings with a **DB parameter group**. The default group can't be modified, so create your own. Its **family** must match the engine version:

```
VER=$(aws rds describe-db-instances --db-instance-identifier lab07b-db \
  --query 'DBInstances[0].EngineVersion' --output text)
FAMILY=$(aws rds describe-db-engine-versions --engine postgres --engine-version $VER \
  --query 'DBEngineVersions[0].DBParameterGroupFamily' --output text)
echo $VER $FAMILY                     # e.g. 17.x postgres17

aws rds create-db-parameter-group --db-parameter-group-name lab07b-pg \
  --db-parameter-group-family $FAMILY --description "lab07b: log slow queries"

aws rds modify-db-parameter-group --db-parameter-group-name lab07b-pg --parameters \
  "ParameterName=log_min_duration_statement,ParameterValue=500,ApplyMethod=immediate" \
  "ParameterName=log_connections,ParameterValue=1,ApplyMethod=immediate"

aws rds modify-db-instance --db-instance-identifier lab07b-db \
  --db-parameter-group-name lab07b-pg --apply-immediately

aws rds describe-db-instances --db-instance-identifier lab07b-db \
  --query 'DBInstances[0].DBParameterGroups' --output table   # ParameterApplyStatus
```

Both parameters are **dynamic**, so changing their values in a group that's already attached takes effect without a reboot. But **switching an instance to a different parameter group** shows `pending-reboot` until you reboot it:

```
aws rds reboot-db-instance --db-instance-identifier lab07b-db
aws rds wait db-instance-available --db-instance-identifier lab07b-db
aws rds describe-db-instances --db-instance-identifier lab07b-db \
  --query 'DBInstances[0].DBParameterGroups' --output table   # now in-sync
```

On the app server:

```
psql -h $PRIMARY -c "SHOW log_min_duration_statement;"     # 500ms
psql -h $PRIMARY -c "SELECT pg_sleep(0.6);"                # slower than 500 ms → logged
```

The slow query now appears in the DB logs (**RDS → lab07b-db → Logs & events**), if your lab lets you open them.

Stretch (only if you have 10+ minutes left): promote the replica.

```
aws rds promote-read-replica --db-instance-identifier lab07b-replica
aws rds wait db-instance-available --db-instance-identifier lab07b-replica
```

Then on the app server, `SELECT pg_is_in_recovery();` on the replica returns `f` and an `INSERT` works. It's now an independent database: it no longer follows the primary, and new rows on one don't appear on the other. Be ready to explain when you'd do this.

A7. Cleanup (required, because RDS bills per hour)

Delete the replica **before** the primary:

```
aws rds delete-db-instance --db-instance-identifier lab07b-replica --skip-final-snapshot
aws rds delete-db-instance --db-instance-identifier lab07b-db \
  --skip-final-snapshot --delete-automated-backups
aws ec2 terminate-instances --instance-ids $APP_ID

aws rds wait db-instance-deleted --db-instance-identifier lab07b-replica
aws rds wait db-instance-deleted --db-instance-identifier lab07b-db
aws rds delete-db-parameter-group --db-parameter-group-name lab07b-pg
aws rds delete-db-subnet-group --db-subnet-group-name lab07b-db-subnets

aws ec2 wait instance-terminated --instance-ids $APP_ID
aws ec2 delete-security-group --group-id $SG_DB     # SG_DB first: it references SG_APP
aws ec2 delete-security-group --group-id $SG_APP
rm -f .dbpw
```

Show `aws rds describe-db-instances` with nothing `lab07b` left, then click **End Lab**.

Track B — no AWS (Docker)

Every command works with `podman` instead of `docker`. Run everything in **bash** (in zsh, the `$P` / `$R` shortcuts below don't split into words), in one folder, e.g. `mkdir lab07b && cd lab07b && bash`.

B1. Network, volumes, passwords

```
docker network create --internal db-private
docker volume create course-db-data
docker volume create replica-data

PW=$(openssl rand -hex 16); RPW=$(openssl rand -hex 16)
printf 'POSTGRES_USER=dbadmin\nPOSTGRES_PASSWORD=%s\n' "$PW" > db.env
printf 'PGHOST=course-db\nPGPORT=5432\nPGUSER=dbadmin\nPGPASSWORD=%s\nPGDATABASE=postgres\n' "$PW"
> app.env
printf 'PGPASSWORD=%s\n' "$RPW" > repl.env
echo "$RPW" > .rpw
unset PW
chmod 600 db.env app.env repl.env .rpw
```

B2. Start the primary and load data

```
docker run -d --name course-db --network db-private --env-file db.env \
  -v course-db-data:/var/lib/postgresql/data postgres:17
until docker run --rm --network db-private --env-file app.env postgres:17 pg_isready -q; do sleep
1; done

# shortcuts: P = psql on the primary, R = psql on the replica
P="docker run --rm --network db-private --env-file app.env postgres:17 psql"
R="docker run --rm --network db-private --env-file app.env -e PGHOST=course-db-replica postgres:17
psql"

$P -c "CREATE TABLE orders (id serial PRIMARY KEY, amount numeric(10,2), created_at timestamptz
DEFAULT now());" \
   -c "INSERT INTO orders (amount) SELECT round((random()*100)::numeric, 2) FROM
generate_series(1, 5000);" \
   -c "SELECT count(*) FROM orders;"      # 5000
```

B3. A replication user

RDS does this for you. Here you do it by hand: a role allowed to stream changes, and a `pg_hba.conf` line that lets it connect.

```
$P -c "CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '$(cat .rpw)';"
docker exec course-db sh -c 'echo "host replication replicator all scram-sha-256" >>
"$PGDATA/pg_hba.conf"'
docker exec course-db psql -U dbadmin -d postgres -c 'SELECT pg_reload_conf();'
```

B4. Create the replica

`pg_basebackup` copies the primary's data into the new volume. `-R` writes `standby.signal` and `primary_conninfo`, so the copy starts as a replica that follows `course-db`.

```
docker run --rm --user postgres --network db-private --env-file repl.env \
  -v replica-data:/var/lib/postgresql/data postgres:17 \
  pg_basebackup -h course-db -U replicator -D /var/lib/postgresql/data -R -X stream -P

docker run -d --name course-db-replica --network db-private \
  -v replica-data:/var/lib/postgresql/data postgres:17
until docker run --rm --network db-private --env-file app.env -e PGHOST=course-db-replica
postgres:17 pg_isready -q; do sleep 1; done
```

B5. Prove it's a replica

Predict each result first:

```
$R -c "SELECT pg_is_in_recovery();"                # t
$R -c "SELECT count(*) FROM orders;"               # 5000
$R -c "INSERT INTO orders (amount) VALUES (1);"    # ERROR: cannot execute INSERT in a read-only
transaction

$P -c "INSERT INTO orders (amount) SELECT 42 FROM generate_series(1, 100);"
$R -c "SELECT count(*) FROM orders;"               # 5100

# lag, seen from the primary
$P -c "SELECT client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"   # streaming
| async
# and from the replica: time since the last replayed change
$R -c "SELECT now() - pg_last_xact_replay_timestamp() AS lag;"
```

B6. Change settings: reload vs restart

`ALTER SYSTEM` plays the role of the parameter group:

```
$P -c "SHOW log_min_duration_statement;"                        # -1 (off)
$P -c "ALTER SYSTEM SET log_min_duration_statement = 500;" \
   -c "ALTER SYSTEM SET log_connections = on;" \
   -c "SELECT pg_reload_conf();"
$P -c "SHOW log_min_duration_statement;"                        # 500ms, no restart needed

$P -c "SELECT pg_sleep(0.6);"
docker logs course-db 2>&1 | grep -E 'duration:|connection authorized' | tail -3   # the slow
query is logged
```

A **static** setting is different: a reload isn't enough.

```
$P -c "ALTER SYSTEM SET shared_buffers = '256MB';" -c "SELECT pg_reload_conf();"
$P -c "SELECT name, setting, pending_restart FROM pg_settings WHERE name = 'shared_buffers';"   #
pending_restart = t
docker restart course-db
until docker run --rm --network db-private --env-file app.env postgres:17 pg_isready -q; do sleep
1; done
$P -c "SHOW shared_buffers;"                                    # 256MB now
```

That `pending_restart = t` is exactly RDS's `pending-reboot`.

Stretch: promote the replica.

```
$R -c "SELECT pg_promote();" -c "SELECT pg_is_in_recovery();"   # f
$R -c "INSERT INTO orders (amount) VALUES (7);"                 # works now
```

B7. Cleanup

```
docker rm -f course-db course-db-replica
docker volume rm course-db-data replica-data
docker network rm db-private
rm -f db.env app.env repl.env .rpw
```

Be ready to explain

1. Read replica vs Multi-AZ: what does each one solve, and which one has its **own endpoint**?
2. Why did the `INSERT` on the replica fail? Where must the app send writes, and where can it send reads?
3. Replication is **asynchronous**. A student saves their registration and immediately reloads the page, which reads from the replica. What might they see, and how would you avoid it?
4. Why does RDS require automated backups to be on before it creates a read replica? (Track B: what did `pg_basebackup` copy?)

5. Dynamic vs static parameter: which of yours applied immediately, and what does `pending-reboot` (or `pending_restart`) mean?
6. What changes when you **promote** a replica, and when would you do it?

Common mistakes

- **`create-db-instance-read-replica` fails: backups not enabled.** The primary needs `--backup-retention-period 1` or more. Changing it later needs a modify and some waiting. Create it right the first time.
- **Waiting and doing nothing (Track A).** The primary and the replica each take minutes. Start them early and do A3 or A6 while they build.
- **The replica "doesn't have the new rows".** Replication is asynchronous. Query again a second later.
- **Writes go to the replica endpoint.** Read replicas are read-only. Point writes at the primary endpoint.
- **`DBParameterGroupFamily` mismatch.** The family must match the engine version (e.g. `postgres17` for 17.x). Look it up with `describe-db-engine-versions`, don't guess.
- **`SHOW` still shows the old value (Track A).** The instance is in `pending-reboot` after switching parameter groups. Reboot it.
- **Cleanup order (Track A).** Delete the replica before the primary, and wait for both to be gone before deleting the parameter group and the subnet group.
- **`$P: command not found` (Track B).** You're in zsh. Run `bash` first, or type the full `docker run ... psql` command.
- **`pg_basebackup` fails with `no pg_hba.conf entry for replication` (Track B).** You skipped the `pg_hba.conf` line in B3, or didn't reload.